UNIT-5

Introduction to Unsupervised learning

Introduction to Unsupervised learning

- Unsupervised Learning is a type of Machine Learning in which the model is trained **without labeled output data**.
- In supervised learning, we have input-output pairs (X, Y).
- In unsupervised learning, we have **only input data** X.

The goal is to:

- Discover hidden patterns (Most customers who buy **bread also buy butter**.)
- Identify structure in data(Grouping or organizing data so that **similar items are together**.)
- Group similar data points.
- Reduce dimensionality (removes unnecessary or duplicate data)
- Estimate probability distributions (determine how values are distributed)

Formally, given a dataset:

$$D = \{x_1, x_2, x_3, . , .. x_n\}$$

Each $x_i \in \mathbb{R}^d$represents a data point with d features, and the goal is to learn the probability distribution $P(X)$that describes the structure of the dataset.

2. Key Characteristics

According to Bishop and Murphy:

- No target variable (no labels)
- Focuses on modeling the probability density function
- Finds hidden structure
- Used for exploratory data analysis

Objectives of Unsupervised Learning

1. Density Estimation

Estimate the probability distribution $P(X)$.

Example:

- Gaussian Mixture Models (GMM)

2. Clustering

Group similar data points into clusters.

Objective:

Minimize *within−cluster variance*

Example:

- K-Means
- Hierarchical Clustering

3. Dimensionality Reduction

Reduce the number of features while preserving important information.

Example:

- Principal Component Analysis (PCA)

4. Association Rule Learning

Discover relationships between variables.

Example:

- Market Basket Analysis (understand which products are frequently purchased together)

Introduction to clustering

Clustering is an **unsupervised learning technique** that aims to group a set of data points into clusters such that:

- Data points within the same cluster are **similar**
- Data points in different clusters are **dissimilar**

Unlike classification, clustering does **not use labeled data**.

Given dataset:

$$D = \{x_1, x_2, \ldots x_n\}, x_i \in \mathbb{R}^d$$

The objective is to partition the dataset into K clusters:

$$C = \{C_1, C_2, \ldots C_K\}$$

such that:

$$C_i \cap C_j = \emptyset \ (i \neq j)$$

2. Objectives of Clustering

According to Bishop and Murphy:

1. Discover Structure

Identify hidden patterns in data.

2. Minimize Intra-cluster Distance

Points inside a cluster should be close.

3. Maximize Inter-cluster Distance

Clusters should be well separated.

Mathematically:

where:

- μ_k =centroid of cluster k

Types of Clustering Methods

1. Partition-Based Clustering

- Divides data into **K non-overlapping clusters**.
- Each data point belongs to exactly one cluster.
- Example: K-Means
- Minimizes within-cluster variance.

2. Hierarchical Clustering

- Creates a **tree-like structure (dendrogram)**.
- Two types:
  - Agglomerative (bottom-up)
  - Divisive (top-down)

3. Density-Based Clustering

- Forms clusters based on **data density**.
- Groups points that are closely packed together.
- If a point is alone or far from others, it may be treated as noise (outlier).
- Handles noise well.
- Example: DBSCAN

4. Model-Based Clustering

- Assumes data is generated from a **probability model**.
- Uses statistical methods.
- Example: Gaussian Mixture Model (GMM)
- Provides soft (probabilistic) clustering.

5. Grid-Based Clustering

- Divides data space into fixed grid cells.
- Clustering performed on grid structure.
- Efficient for large datasets.
- Cells with many points form clusters.
- Cells with few or no points are ignored.

Hierarchical Clustering

2 Types:

A. Agglomerative Hierarchical Clustering (AHC)

Also called **Bottom-Up Approach**.

Algorithm:

1. Start with each data point as a separate cluster.
2. Compute pairwise distances.
3. Merge the two closest clusters.
4. Update distance matrix.
5. Repeat until all points form one cluster.

This produces a nested sequence of merges.

B. Divisive Hierarchical Clustering

Also called **Top-Down Approach**.

Algorithm:

1. Start with all points in one cluster.
2. Split the cluster into two.
3. Recursively split clusters.
4. Continue until each point becomes its own cluster.

Divisive methods are computationally more expensive.

Agglomerative Clustering

- Agglomerative Clustering is a **hierarchical clustering algorithm** that builds a hierarchy (tree) of clusters by **iteratively merging the closest pair of clusters**.
- Unlike partitioning methods like *k-means*, it does **not require the number of clusters in advance**.
- Instead, it produces a **dendrogram** that illustrates how clusters merge at different levels of similarity or distance.

1. Initialization:

1. Start with **each data point as its own cluster**.

2. Compute Distances:

1. Calculate a **distance (or similarity) matrix** between all clusters.

3. Merge Clusters:

1. Locate the **two closest clusters** and merge them into a new cluster.

4. Update Distances:

1. Recompute distances between the new cluster and all existing clusters using a **linkage criterion**.

5. Repeat:

1. Continue merging until:
    1. All points form a single cluster, or
    2. A stopping criterion is met (e.g., desired number of clusters).

Linkage Criteria (How to Measure Cluster Distance)

- The way distances between clusters are defined greatly affects the clustering result.

Linkage Type	Definition
Single Linkage	Minimum distance between points of two clusters. (Clusters join if at least two points are very close.)
Complete Linkage	Maximum distance between points of two clusters.(Clusters merge only if all points are relatively close.)
Average Linkage	Average pairwise distance between cluster points.(Uses the average distance instead of only the closest or farthest points.)
Centroid Linkage	Distance between cluster centroids.(If two cluster centers are close, the clusters may merge.)
Ward's Method	Minimizes increase in total within-cluster variance(**the pair of clusters whose merge causes the smallest increase in total variance**.)

Imagine grouping **students based on marks**.

Students:
40, 42, 45, 80, 82

Clusters merge like:

(40,42) - small difference
(80,82) - small difference

Then:

(40,42,45)

Finally two big clusters merge.

Distance Metrics

Before computing linkage, a **distance metric** must be chosen:

- **Euclidean Distance**
- **Manhattan Distance**
- **Cosine Distance**

Different metrics yield different cluster shapes and results.

Mathematical Intuition

Let:

- $X = \{x_1, x_2, ., .. x_n\}$
- $d(A, B)$be the distance between clusters A and B

At each iteration:

$$(A^*, B^*) = \arg\min_{A,B} \quad d(A, B)$$

Where linkage function d is defined per the chosen criterion.

For **Ward's linkage**:

$$d(A, B) = \frac{|A| \quad |B|}{|A| + |B|} \| \bar{x}_A - \bar{x}_B \|^2$$

Where $\bar{x}_A$and $\bar{x}_B$are centroids.

|A| and |B|

These mean:

- |A| = number of points in cluster A
- |B| = number of points in cluster B

$\bar{x}_A$ and $\bar{x}_B$

These are:

Centroids (means) of clusters

Centroid = average of all points inside the cluster.

Example:

If cluster A has points:
(2,3), (4,5)

Centroid is: $\left(\frac{2+4}{2}, \frac{3+5}{2}\right) = (3, 4)$

$\|\bar{x}A - \bar{x}B\|2$

This means:

Squared distance between the two centroids.

So Ward's method checks:

- How far apart are the cluster centers?

The Fraction Part

$$\frac{|A||B|}{|A|+|B|}$$

This is a **weighting factor**.

It ensures:

•Bigger clusters influence the result more.

•The merging does not create imbalance.

Ward's distance measures:

How much the total cluster variance will increase if we merge A and B.

So the algorithm chooses:

- The merge that causes the **smallest increase in variance**
- Which results in **compact and balanced clusters**

Suppose:

- Total data points = 10
- Desired clusters = 3

Process:

The loop stops when:

number of clusters == desired

Step	Number of Clusters
Start	10
Merge 1	9
Merge 2	8
Merge 3	7
Merge 4	6
Merge 5	5
Merge 6	4
Merge 7	3 → STOP

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage
```

 # Step 1: Load Dataset

```
df =  pd.read_csv(r"C:\Users\StudentsPerformance.csv")
```

 # Step 2: Select only numeric score columns

```
X = df[["math score", "reading score", "writing score"]]
```

 # Step 3: Standardize the data

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

`transform()` converts values using the formula:

$$Z = \frac{X - Mean}{StandardDeviation}$$

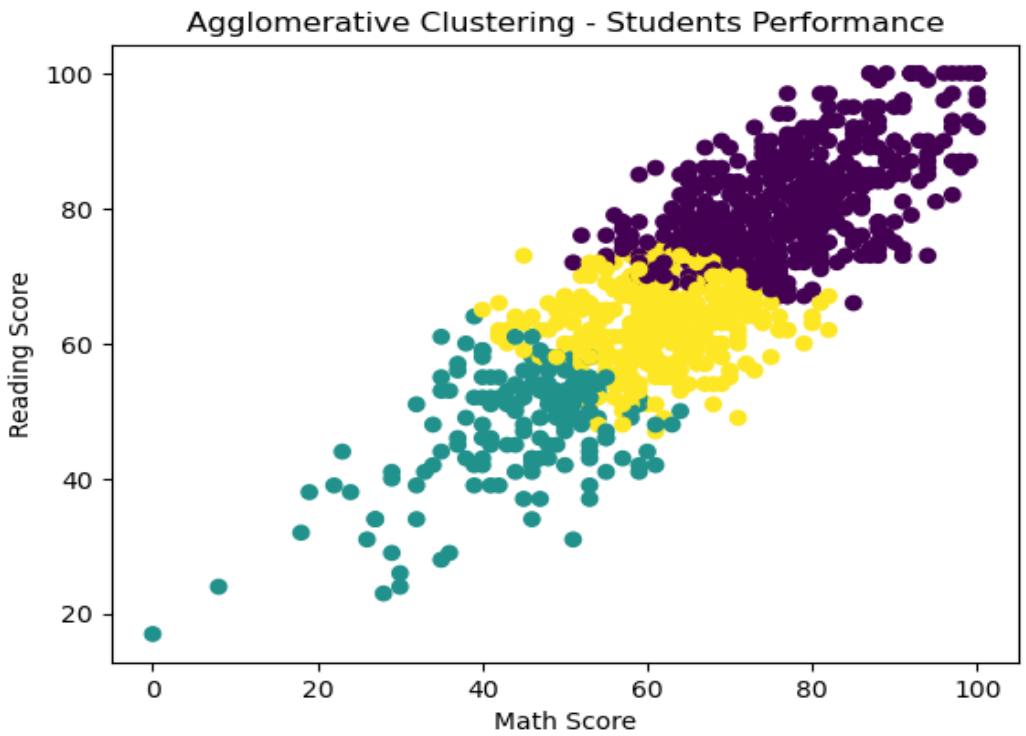
Where:

X = original value

Mean = average

Standard deviation = data spread

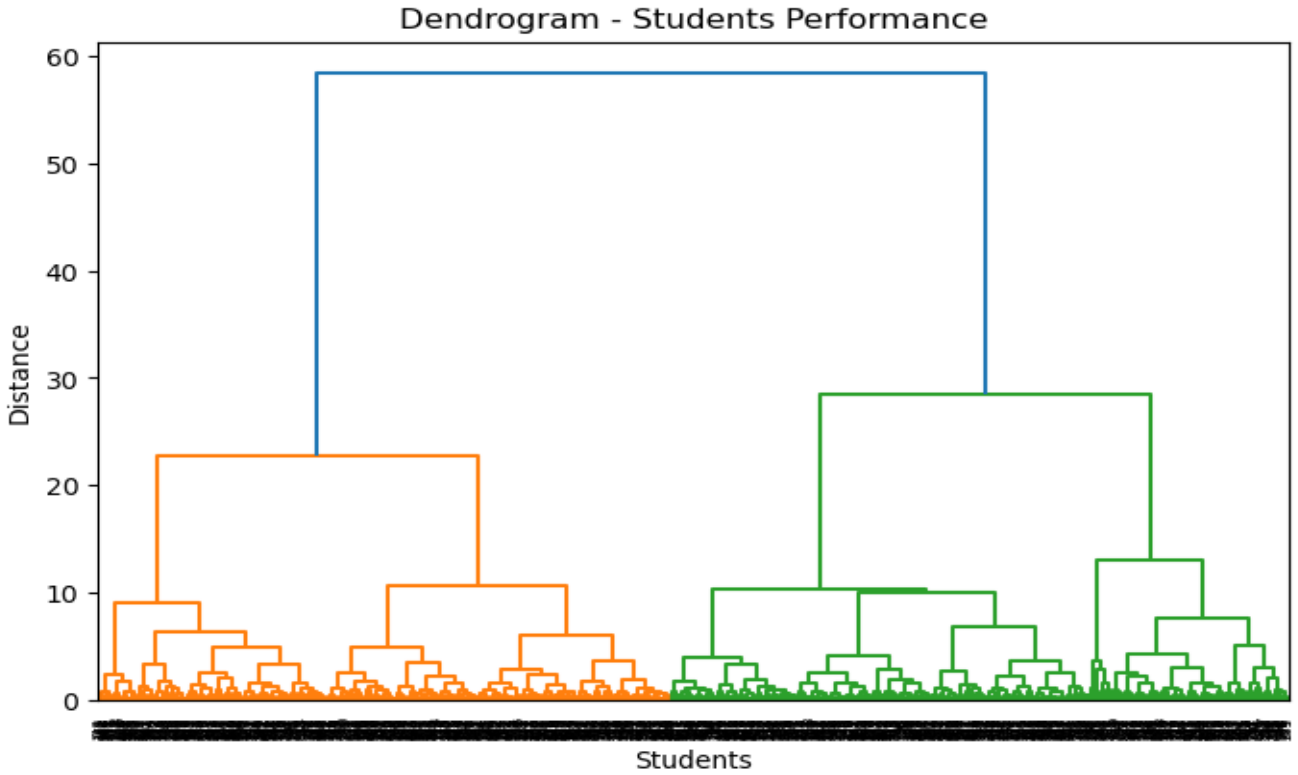
```
# Step 4: Apply Agglomerative Clustering
model = AgglomerativeClustering(n_clusters=3, linkage='ward')
labels = model.fit_predict(X_scaled)

# Step 5: Add cluster labels to dataset
df["Cluster"] = labels

print("\nClustered Data:")
print(df[["math score", "reading score", "writing score", "Cluster"]].head())

# Step 6: Visualize (Math vs Reading)
plt.scatter(df["math score"], df["reading score"], c=labels)
plt.xlabel("Math Score")
plt.ylabel("Reading Score")
plt.title("Agglomerative Clustering - Students Performance")
plt.show()
```



```
linked = linkage(X_scaled, method='ward')

plt.figure(figsize=(8,5))
dendrogram(linked)
plt.title("Dendrogram - Students Performance")
plt.xlabel("Students")
plt.ylabel("Distance")
plt.show()
```

- which clusters merged
- distance between clusters
- number of points in cluster

This is later used to **draw a dendrogram**.



Divisive clustering

- Divisive clustering is a **hierarchical clustering technique** that follows a **top-down approach**.
- Unlike agglomerative clustering (which starts with individual data points and merges them), divisive clustering:
- **Starts with all data points in one single cluster and recursively splits them into smaller clusters.**
- Thus, it builds a hierarchy by **division instead of merging**.

Basic Idea

Given dataset:

$$X = \{x_1, x_2, . , .. x_n\}$$

Step 1:

Place all points in one cluster.

Step 2:

Select a cluster to split.

Step 3:

Divide it into two smaller clusters using a partitioning method.

Step 4:

Repeat splitting until:

- Each data point becomes its own cluster, or
- Desired number of clusters is reached.

Algorithmic Procedure

1. Start with cluster $C = X$
2. While stopping criterion not met:
    1. Select cluster C_i to split
    2. Partition C_i into C_{i1} and C_{i2}
    3. Replace C_i with C_{i1}, C_{i2}

Divisive clustering does not define one fixed splitting rule. Common strategies include:

Using k-means (k = 2)

Split the selected cluster into two subclusters.

Maximizing cluster dissimilarity

Split points that are most dissimilar from others.

DIANA Algorithm

(Divisive Analysis Clustering)

DIANA:

- Finds the object most dissimilar from the rest
- Creates a new cluster
- Iteratively moves similar objects into that new cluster

Mathematical Intuition

Let cluster C contain points:

$$C = \{x_1, x_2, . , .. x_m\}$$

We aim to split C into:

$$C_1 \cup C_2 = C$$

Such that:

- Intra-cluster similarity is high
- Inter-cluster similarity is low

Often achieved by minimizing:

$$\text{Within−cluster } \textit{variance}$$

Similar to k-means objective:

Where:

- μ_i is centroid of cluster C_i

Dendrogram Representation

Divisive clustering also produces a **dendrogram**, but it grows from:

Top → Down

Comparison with Agglomerative Clustering

Aspect	Agglomerative	Divisive
Approach	Bottom-Up	Top-Down
Start	n clusters	1 cluster
Operation	Merge	Split
Computational Cost	Moderate	Higher
Common Usage	Very common	Less common

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage

df =  pd.read_csv(r"C:\Users\StudentsPerformance.csv")

# Select score columns
X = df[["math score", "reading score", "writing score"]]

# Standardize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Divisive Clustering (Top-Down Split into 2 clusters)
kmeans = KMeans(n_clusters=2, random_state=42)
df["Cluster"] = kmeans.fit_predict(X_scaled)

# Display first 5 rows
print(df[["math score", "reading score", "writing score",
 "Cluster"]].head())

# Visualize
plt.scatter(df["math score"], df["reading score"], c=df["Cluster"])
plt.xlabel("Math Score")
plt.ylabel("Reading Score")
plt.title("Divisive Clustering - Students Performance")
plt.show()

# Create linkage matrix (Ward's method)
linked = linkage(X_scaled, method='ward’)

# Plot dendrogram
plt.figure(figsize=(10, 5))
dendrogram(linked)
plt.title("Dendrogram - Students Performance Dataset")
plt.xlabel("Students")
plt.ylabel("Distance")
plt.show()
```

Partial Clustering

- Partial Clustering is a clustering approach in which **only meaningful subsets of data are grouped into clusters**, while some data points remain unclustered (treated as noise or outliers).
- Unlike partitioning methods such as K-Means that assign every point to a cluster, partial clustering identifies only strong and dense patterns.

Purpose of Partial Clustering

In real-world datasets:

- Some data points are noisy.
- Some points do not clearly belong to any group.
- Forcing all points into clusters reduces accuracy.

Partial clustering:

- Forms only strong clusters
- Leaves noise unclustered

Formal Concept

Let the dataset be:

$$D = \{x_1, x_2, x_3, ., .. x_n\}$$

Traditional clustering partitions the entire dataset into k clusters:

$$C = \{C_1, C_2, ., .. C_k\}$$

In **partial clustering**, only a subset $D' \subseteq D$is clustered:

$$D' = \bigcup_{i=1}^{k} C_i$$

Such that:

$$D' \neq D$$

The remaining points:

$$D - D'$$

are considered:

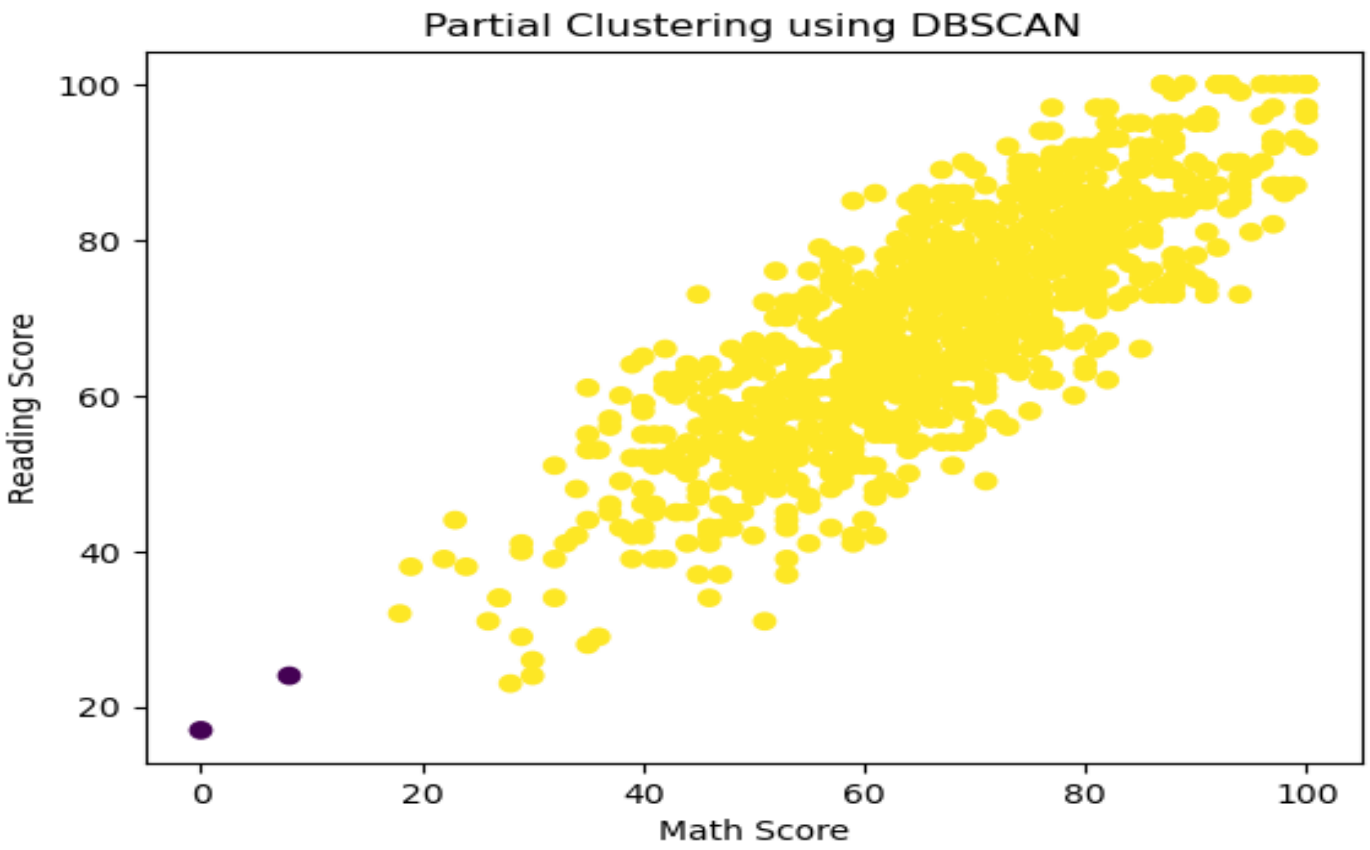
- Noise
- Outliers
- Unclustered data

C_1, C_2, C_3 … C_k → Clusters.
k → Total number of clusters.
∪ (Union) → Combine all clusters together.

Characteristics of Partial Clustering

1. Does not force assignment of all data points.
2. Identifies only strong and meaningful clusters.
3. Handles noise effectively.
4. Often used in density-based and subspace clustering.
5. Produces more reliable cluster structures in noisy datasets.

```
# PARTIAL CLUSTERING USING DBSCAN
# Students Performance Dataset Example

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN

# Step 1: Load Dataset
df =  pd.read_csv(r"C:\StudentsPerformance.csv")

# Step 2: Select Features (Math & Reading scores)
X = df[['math score', 'reading score']]

# Step 3: Apply DBSCAN (Partial Clustering)
model = DBSCAN(eps=8, min_samples=5)
df['Cluster'] = model.fit_predict(X)

# Step 4: Display first few rows
print("Sample Output:")
print(df[['math score', 'reading score',
'Cluster']].head())

# Step 5: Count clusters
print("\nCluster Counts:")
print(df['Cluster'].value_counts())

# Step 6: Visualization
plt.scatter(df['math score'], df['reading score’],
c=df['Cluster'])
plt.xlabel("Math Score")
plt.ylabel("Reading Score")
plt.title("Partial Clustering using DBSCAN")
plt.show()
```


Partial Clustering using DBSCAN
Reading Score
Math Score
100
80
60
40
20
0
20
40
60
80
100

Difference Between Partial Clustering (DBSCAN) and K-Means Clustering

K-Means Clustering

- K-Means is a **partition-based clustering algorithm** that divides a dataset into *K predefined clusters* by minimizing the within-cluster sum of squares (WCSS).

WCSS=∑(distance between data point and its cluster centroid)2

We square the distance because:

- Removes **negative values**
- Gives **more penalty for large distances**
- Helps clusters stay **compact**
- Good Clustering
  Points are **close to the centroid → Low WCSS**

Partial Clustering (DBSCAN)

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a **density-based clustering algorithm** that forms clusters based on dense regions of data and labels sparse points as noise.
- It is called **partial clustering** because it does not force every data point into a cluster.

Requirement of Number of Clusters

K-Means

- Requires the user to specify **K (number of clusters)** in advance.
- If K is wrongly chosen, clustering quality decreases.

DBSCAN

- Does **not require number of clusters**.
- Automatically determines clusters based on:
  - `eps` (neighborhood radius)
  - `min_samples` (minimum points in dense region)

Handling of Noise and Outliers

K-Means

- Does **not handle outliers well**.
- Every point is assigned to the nearest cluster, even if it is far away.

DBSCAN

- Explicitly detects **noise points**.
- Labels them as **-1**.
- Better for real-world noisy datasets.

Cluster Shape

K-Means

- Produces **circular or spherical clusters**
- Poor performance when clusters are non-linear

DBSCAN

- Can detect:
  - Irregular shapes
  - Non-linear boundaries
  - Nested clusters

Advantages

K-Means

- Simple and easy to implement
- Works well for compact clusters
- Fast for large datasets

DBSCAN

- Detects noise automatically
- No need to specify number of clusters
- Finds arbitrary shaped clusters

Limitations

K-Means

- Requires K in advance
- Sensitive to initial centroid selection
- Not suitable for non-spherical clusters

DBSCAN

- Sensitive to parameter selection (eps, min_samples)
- Struggles with varying density clusters